

UNIVERSITÀ DEGLI STUDI ROMA TRE

Dipartimento di Ingegneria Civile, Informatica e delle Tecnologie Aeronautiche

Corso di Laurea in Ingegneria Informatica

Tesi di Laurea

Valutazione sperimentale della memoria persistente negli LLM per la personalizzazione di assistenti conversazionali

Laureando

Giorgio Lai

Matricola 610105

Relatore

Prof. Fabio Gasparetti

Anno Accademico 2025/2026

"Condividete la vostra energia con me,

fino al limite del possibile!"

— Son Goku, Dragon Ball

Ringraziamenti

Vorrei cominciare questi ringraziamenti partendo dalle più importanti persone che mi sono sempre state vicine fin da sempre ossia la mia famiglia.
Vorrei ringraziare mia madre e mio padre del loro supporto che mi hanno aiutato per tutta la mia vita aiutandomi ad affrontare i miei problemi e di come crescere come persona.

Vorrei ringraziare mia madre Helen di avermi aiutato nell'affrontare i miei momenti di difficoltà facendo dei sacrifici che io non conosco ancora tutt'oggi permettendomi di alleggerire la mente per studiare, divertirmi e farmi diventare una persona con dei valori. Vorrei ringraziare mio padre Francesco di avermi aiutato nei miei momenti difficoltà sia grandi che piccoli anche quando non chiedevo aiuto e di avermi fatto crescere come persona con dei valori.

Vorrei ringraziare mio fratello Antonio che mi ha aiutato ad essere una linea guida per i miei obiettivi facendomi scoprire da sempre interessi che ad oggi guardo sempre. Senza di lui non avrei mai avuto cominciato ad avere interesse nel mondo dell'informatica consigliandomi la mia scuola superiore. Quindi se oggi sono un laureato in ingegneria informatica il merito va in buona parte a lui.

Grazie mille per tutto alla mia famiglia.

Vorrei ringraziare i miei amici di aver passato questi anni molto belli in cui se ricordo certi momenti per cui rido ancora anche se mi dispiace di non essere sempre stato con voi dovendo mettere da parte il divertimento per inseguire i miei sogni.

Desidero infine dedicare un ultimo ringraziamento ai miei che ho conosciuto durante questi 3 anni di università che mi hanno aiutato ad affrontare i diversi ostacoli del mio percorso di studi, questo mi ha aiutato a capire che aiutarsi a vicenda permette a tutti di affrontare i nostri problemi. In fondo l'unione fa la forza. Nel dettaglio voglio ringraziare le seguenti persone: Riccardo Cacciamani, Simone infantino, Giorgio Lauria, Adriano Riccobono, Daniele Panzeri, Mattia Lenti, Chiara Brancaleoni, Simone Improta.

Introduzione

I modelli linguistici vengono utilizzati in varie attività che possono essere quelle più semplici come una discussione su un argomento di attualità tra una persona e l'assistente fino all'utilizzo di progetti complessi, quindi i modelli linguistici hanno impattato molto sul modo di fare delle persone semplificando alcuni compiti più complessi e ripetitivi.

I modelli linguistici per riuscire a rispondere alle domande che ricevono dall'utente oltre a dipendere dai parametri addestrati, utilizzano la finestra di contesto che gli permette di memorizzare temporaneamente le informazioni che gli sono state fornite ingresso.

La finestra di contesto ha una capacità limitata nel possedere le informazioni nel tempo ciò porta al modello linguistico di memorizzare solo le ultime informazioni che l'utente gli fornisce nella discussione. Questo porta al modello linguistico alla perdita di informazioni che possono essere importanti ai fini della generazione delle risposte causando delle possibili allucinazioni che sono delle risposte che non sono pertinenti a ciò che l'utente desidera avere. Un altro problema legato sempre alla finestra di contesto è che l'utente deve rimandare sempre le stesse informazioni precedenti nella discussione che sta avendo con il modello linguistico causando quindi un dispendio di token di utilizzo del modello linguistico.

Nel corso degli anni sono stati sviluppati dei sistemi di memoria che permettono di risolvere i limiti della finestra di contesto attraverso diversi approcci facendo avere così all'utente l'impressione che i modelli linguistici possiedono la capacità di ricordare le

informazioni passate.

L'obiettivo della presente tesi è quello di sperimentare le diverse idee che sono state create da alcuni autori nel corso degli anni dei diversi sistemi di memoria. Per la realizzazione degli esperimenti della presente tesi sono state fissate le seguenti domande di ricerca:

- RQ1 — Effetto della disponibilità della memoria. In che misura la disponibilità di informazioni provenienti da sessioni precedenti migliora la correttezza delle risposte?
- RQ2 — Effetto della rappresentazione della memoria. In che misura impatta la rappresentazione della memoria sul recupero delle informazioni rilevanti e sulla correttezza delle risposte?
- RQ3 — Effetto della condivisione della memoria. Quale effetto produce la condivisione della memoria tra attività differenti sul recupero delle informazioni rilevanti e sulla correttezza delle risposte?
- RQ4 — Continuità dopo il cambio del modello. Quali differenze emergono nella continuità conversazionale quando il modello generativo viene sostituito utilizzando una memoria separata oppure condivisa?
- RQ5 — Effetto del modello generativo. A parità di informazioni, quali differenze emergono tra i modelli quando le informazioni sono sufficienti e nella capacità di astenersi quando sono insufficienti?

Nella presente tesi, è stato scelto come dominio applicativo la sicurezza informatica perché è uno dei settori in cui molte attività richiedono di mantenere le informazioni distribuite nel tempo. L'insieme dei dati della sicurezza informatica è stato utilizzato per progettare degli scenari sintetici in cui sono dei dialoghi tra l'utente e l'assistente che non

rappresentano delle conversazioni aziendali reali, nel dettaglio alcuni scenari sono stati costruiti su dati fittizi mentre altri su dati reali trovati su fonti esterne.

Per realizzare tali esperimenti è stato costruito un apposito framework in locale in cui fa da intermediario tra l'utente e il modello linguistico. Tale framework possiede dei sistemi di memoria che sono dei adattamenti semplificati dei principali lavori illustrati nello stato dell'arte. Per valutare gli esperimenti sono state utilizzate delle metriche, tabelle e grafici per valutarne delle conclusioni.

Il contributo principale della tesi consiste nella progettazione e implementazione di un framework sperimentale per confrontare diverse configurazioni di memoria conversazionale, valutando l'effetto della disponibilità, rappresentazione e condivisione della memoria sulla capacità degli LLM di recuperare informazioni pregresse e produrre risposte coerenti.

La presente tesi è stata suddivisa nei seguenti capitoli: il capitolo del dominio di riferimento ha l'obiettivo di introdurre il contesto su cui si basa l'intera tesi e di quali problematiche affronterà, il capitolo di background ha l'obiettivo di fornire delle definizioni sugli strumenti che vengono utilizzati nell'intera tesi, il capitolo dello stato dell'arte ha l'obiettivo di illustrare quali lavori hanno fornito i principi che si basano l'intera tesi, il capitolo del framework sperimentale ha l'obiettivo di illustrare come è stato creato l'intero framework sperimentale su cui è sono stati eseguiti gli esperimenti, il capitolo della progettazione degli esperimenti ha l'obiettivo di illustrare come sono stati progettati tutti gli esperimenti, il capitolo dei risultati ha l'obiettivo di spiegare i risultati forniti dagli esperimenti eseguiti, infine le conclusioni hanno l'obiettivo di riassumere l'intero lavoro, quali sono stati gli aspetti soddisfacenti e non ed infine i possibili sviluppi futuri.

Indice

Introduzione **v**

Indice **viii**

Elenco delle figure **xii**

1 Dominio di interesse **1**

1.1 Utilizzo e limiti della finestra di contesto 1

1.2 Sviluppo dei sistemi di memoria 2

1.3 Obiettivi del presente lavoro 3

2 Background **4**

2.1 Large Language Model 4

2.2 Memoria per gli LLM 5

2.3 Recupero dell'informazione e RAG 6

2.4 Architetture di memoria esterna 7

2.4.1 Turn-level RAG 8

2.4.2 Fact-based RAG 9

2.4.3 Gestione dinamica della memoria 11

2.4.4 Memoria basata su grafi 12
2.4.5 Memoria gerarchica 13
2.5 Valutazione di un sistema di memoria e metriche 15
2.5.1 Recall@k 16
2.5.2 Exact Match 16
2.5.3 Correct Abstention Rate 16

3 Stato dell'arte **17**
3.1 Tecniche di gestione del contesto 17
3.1.1 Cronologia completa 18
3.2 Sistemi di memoria conversazionali esistenti 20
3.2.1 MemGPT 20
3.2.2 Mem0 e la sua estensione a grafo Mem0g 22
3.2.3 A-MEM 24
3.3 Sintesi critica e posizionamento del presente lavoro 26

4 Framework sperimentale **28**
4.1 Obiettivo 28
4.1.1 Problemi 28
4.1.2 Finalità 29
4.1.3 Che cosa osservare 29
4.2 Funzionamento generale 30
4.2.1 Memorizzazione e gestione delle informazioni 31
4.2.2 Recupero e costruzione del contesto 32
4.2.3 Generazione e tracciabilità della risposta 32

4.3 Forme di memoria considerate 34
4.3.1 Turn-level RAG 34
4.3.2 Fact-based RAG 34
4.3.3 Memoria con aggiornamenti 35
4.3.4 Memoria a grafo 35
4.3.5 Memoria gerarchica 35
4.3.6 Cronologia completa 35

5 Progettazione degli esperimenti 36

5.1 RQ1 — Effetto della disponibilità della memoria 36
5.1.1 Disegno dell'esperimento 36
5.1.2 Configurazione sperimentale 38
5.1.3 Valutazione 39
5.2 RQ2 — Effetto della rappresentazione della memoria 40
5.2.1 Disegno dell'esperimento 40
5.2.2 Configurazione sperimentale 42
5.2.3 Valutazione 42
5.3 RQ3 — Effetto della condivisione della memoria 42
5.3.1 Disegno dell'esperimento 43
5.3.2 Configurazione sperimentale 43
5.3.3 Valutazione 43
5.4 RQ4 — Continuità dopo il cambio del modello 43
5.4.1 Disegno dell'esperimento 44
5.4.2 Configurazione sperimentale 44
5.4.3 Valutazione 45

5.5 RQ5 — Effetto del modello generativo 45
5.5.1 Disegno dell'esperimento 45
5.5.2 Configurazione sperimentale 45
5.5.3 Valutazione 46

6 Risultati sperimentali **47**
6.1 Risultati della prima domanda di ricerca 47
6.1.1 Risultati osservati 47
6.1.2 Risposta alla domanda di ricerca 48
6.2 Risultati della seconda domanda di ricerca 49
6.2.1 Risultati osservati 49
6.2.2 Risposta alla domanda di ricerca 53
6.3 Risultati della terza domanda di ricerca 54
6.4 Risultati della quarta domanda di ricerca 54
6.5 Risultati della quinta domanda di ricerca 55

Conclusioni **56**

Appendice **59**
Comandi da eseguire nel terminale 59
Primo esperimento 59
Secondo esperimento 59
Terzo esperimento 60
Quarto esperimento 60
Quinto esperimento 60

Bibliografia **61**

Elenco delle figure

4.1 Come è strutturato il progetto 30

6.1 Confronto nello scenario 2 49

6.2 Confronto nello scenario 3 50

6.3 Confronto nello scenario 4 51

6.4 Confronto nello scenario 5 52

Capitolo 1

Dominio di interesse

Questo capitolo ha l'obiettivo di fare una panoramica sul contesto dell'utilizzo della memoria conversazionale dei modelli linguistici sia al livello storico che al livello concettuale.

1.1 Utilizzo e limiti della finestra di contesto

Attraverso l'avvento dei modelli linguistici, molte persone usano spesso i modelli linguistici per instaurare delle conversazioni che sono di qualsiasi tipo che vanno dagli ambiti giornalieri fino a quelli più specialistici. I modelli linguistici per ricordare la conversazione con l'utente utilizza la finestra di contesto in cui memorizza brevemente le informazioni che l'utente ha scritto nel messaggio che ha appena ricevuto per poi rispondergli. Il problema della finestra di contesto è che è limitata, ciò può far avere al modello linguistico delle carenze nel ricordare cosa ha detto l'utente per l'intera conversazione costringendo l'utente a rimandare gli stessi messaggi per fare sapere al modello delle informazioni per rispondere alle domande.

Quindi le risposte dei modelli linguistici dipendono soltanto dai parametri addestrati

e dai messaggi ricevuti in ingresso dall'utente. Costringere l'utente di rinviare spesso gli stessi messaggi in diverse sessioni succede che l'utente consuma i token disponibili per usare il modello linguistico.

Secondo Zhang et al. [1] all'inizio i primi agenti combinavano lo storico dei messaggi delle varie sessioni in un unico messaggio da mandare al modello linguistico.

1.2 Sviluppo dei sistemi di memoria

Per sistemare il problema della finestra di contesto sono stati sviluppati dei sistemi di memoria esterna per memorizzare le informazioni dell'utente.

Secondo Zhang et al. [1], i modelli linguistici sono stati dotati delle componenti esterne per percepire l'ambiente circostante ed imparare dall'esperienza quindi utilizzando l'apprendimento per rinforzo arrivando alla fine a dei sistemi di memoria avanzati simili ad un sistema operativo.

Il documento [1] enfatizza che la memoria è l'aspetto più importante per separare un agente autonomo da un modello linguistico.

1.3 Obiettivi del presente lavoro

L'obiettivo della presente tesi è quello di sperimentare vari effetti della memoria conversazionale attraverso delle diverse configurazioni. Le domande di ricerca sono:

- RQ1 — Effetto della disponibilità della memoria. In che misura la disponibilità di informazioni provenienti da sessioni precedenti migliora la correttezza delle risposte?
- RQ2 — Effetto della rappresentazione della memoria. In che misura impatta la rappresentazione della memoria sul recupero delle informazioni rilevanti e sulla correttezza delle risposte?
- RQ3 — Effetto della condivisione della memoria. Quale effetto produce la condivisione della memoria tra attività differenti sul recupero delle informazioni rilevanti e sulla correttezza delle risposte?
- RQ4 — Continuità dopo il cambio del modello. Quali differenze emergono nella continuità conversazionale quando il modello generativo viene sostituito utilizzando una memoria separata oppure condivisa?
- RQ5 — Effetto del modello generativo. A parità di informazioni, quali differenze emergono tra i modelli quando le informazioni sono sufficienti e nella capacità di astenersi quando sono insufficienti?

Capitolo 2

Background

Per capire le tecnologie utilizzate in questa tesi bisogna avere le fondamenta teoriche del loro funzionamento, quali limiti importanti dei modelli linguistici permettono di superare e di come vengono valutate tali tecnologie.

2.1 Large Language Model

Un large language model (LLM) è un modello linguistico di grandi dimensioni in cui è basato sulle reti neurali secondo Minaee et al. [2]. Questi modelli vengono addestrati su grandi raccolte di dati, come ad esempio dei file JSON, chiamate dataset in cui tale addestramento effettuato sui parametri del modello serve per far ottenere ai modelli la capacità di comprendere e generare il linguaggio.

Per elaborare il testo i modelli linguistici utilizzano il meccanismo di auto attenzione in cui fa ottenere dei collegamenti tra le parole. Quando un modello riceve in ingresso un testo, tale testo viene convertito in unità chiamate token che sono dei codici univoci.

Un problema che hanno i modelli linguistici secondo il seguente lavoro [2] è che non

riescono a tenere traccia delle informazioni tra sessioni e per fare ciò bisogna utilizzare un sistema esterno in cui memorizza l'informazione durante le sessioni e nel momento in cui serve al modello linguistico, tale modello recupera l'informazione dal sistema esterno.

2.2 Memoria per gli LLM

La memoria, come descritto da Zhang et al. [1], viene definita in due modi:

- la memoria riguarda i passaggi di un agente nel completare un compito;
- la memoria riguarda un grande insieme di informazioni che possono essere una sequenza di azioni, tentativi diversi e informazioni esterne.

Secondo Zhang et al. [1] la memoria ha le seguenti rappresentazioni:

- nella forma testuale (detta anche forma non parametrica) memorizza le informazioni in linguaggio naturale in un archivio di documenti che è più veloce ed affidabile ma è limitato dalla finestra di contesto;
- nella forma parametrica, attraverso il Supervised fine-tuning oppure Knowledge Editing viene ottenuta l'informazione nei parametri del modello. Questa rappresentazione permette di avere più densità informativa tuttavia ha più costi di lettura;

Le operazioni fondamentali, descritte in [1], che si possono eseguire nella memoria sono: scrittura, lettura e gestione. L'operazione di gestione permette di elaborare la memoria attraverso la generazioni di concetti astratti, l'eliminazione dellle informazioni ridondanti e la cancellazione di dati obsoleti o superflui.

la memoria [1] è necessaria perché permette all'agente sia di apprendere, simulando la mente umana, e sia di adattarsi ad ambienti dinamici sfruttando come principio il reinforcement learning in cui principalmente si utilizza un modulo di memoria.

2.3 Recupero dell'informazione e RAG

Il RAG (Retrieval-Augmented Generation), secondo Lewis et al. [3], è un paradigma che combina un modello generativo con un meccanismo di recupero di informazioni da una sorgente esterna. Le informazioni recuperate vengono fornite al modello come contesto aggiuntivo durante la generazione della risposta.

Il funzionamento del RAG, secondo gli autori [3], è basato su due passaggi fondamentali:

- quando il LLM riceve la domanda in ingresso per prima cosa analizza l'archivio di documenti per poi estrarre le informazioni più importanti;
- il modello legge la domanda ricevuta in ingresso ed attraverso le informazioni più pertinenti ottenute dall'archivio di documenti genera la risposta alla domanda.

La parte importante, secondo gli autori di [3], è che questi due passaggi non lavorano da soli bensì questi moduli vengono addestrate insieme per collaborare nel modo migliore possibile.

Questo metodo [3] è importante perché serve per mitigare i limiti dei modelli per i seguenti motivi:

- basandosi sull'archivio di documenti, il modello inventa meno le informazioni e diventa più preciso quindi riduce le allucinazioni;
- per aggiornare la memoria del modello bisogna solo aggiornare l'archivio di documenti;

- è più efficiente nell'affrontare i compiti più complessi dove richiede una conoscenza più precisa dei dati;
- dato che tale approccio si basa sui testi in linguaggio naturale permette di verificare meglio le fonti.

2.4 Architetture di memoria esterna

L'architettura di memoria è una componente di un modello che si occupa di rielaborare, salvare e riutilizzare le informazioni della sua esperienza nel mondo esterno come descritto da Zhang et al. [1]. L'architettura di memoria aiuta a definire come le informazioni passano tra l'ambiente esterno, la componente di memoria e nel modello linguistico. I flussi di lavoro di una architettura di memoria vengono strutturati in questo modo da Zhang et al. [1]:

- conserva le informazioni di un compito in esecuzione, tale flusso di lavoro viene chiamato memoria di breve termine;
- vengono organizzati i successi, fallimenti e strategie trovate nell'esecuzione dei compiti passati che aiuteranno, evitando quindi che la rete neurale venga di nuovo addestrata sulle stesse informazioni, all'agente di evolversi nel tempo. Tale flusso di lavoro si chiama memoria a lungo termine;
- l'agente si connette a fonti esterne come può essere ad esempio una base di dati. Tale fluso di lavoro si chiama conoscenza esterna.

Secondo Zhang et al. [1] in una prospettiva applicativa ed ingegneristica, quando la memoria viene utilizzata in un sistema concreto viene organizzata in una struttura a livelli

che è ispirata alla psicologia cognitiva oppure ai sistemi operativi. La memoria di lavoro è un parte del contesto in cui il modello può utilizzare subito per generare la risposta. Quindi l'architettura di memoria serve per:

- evitare che un agente non impara dai propri errori;
- evitare che un agente si dimentichi cosa stava facendo prima di una interazione lunga;
- permettere all'agente di ricordare enormi quantità di dati senza rallentare o usare troppa potenza di calcolo.

2.4.1 Turn-level RAG

il Turn-level RAG, come descritto da Wu et al. [4], è una architettura di memoria in cui a differenza dei RAG classici che memorizzano l'intera discussione con l'utente in un unico testo, Turn-level RAG prende l'intera discussione è la suddivide in turni che sono interazioni tra l'utente e il modello.

Il funzionamento dell'architettura [4] è il seguente:

- l'intera discussione viene frammentata in turni che vengono memorizzati in una struttura dati che utilizza il formato chiave-valore;
- quando l'utente fa una nuova domanda, il sistema cerca nell'archivio di documenti i singoli turni più utili che verranno utilizzati per rispondere all'utente;
- dopo che i turni sono stati recuperati dal sistema, il sistema passa questi turni al modello per generare la risposta della domanda era stata passata dall'utente.

Secondo Wu et al. [4] Turn-level RAG è utile perché riesce a risolvere alcuni dei problemi delle discussioni più lunghe come possono essere:

- salvando le informazioni in blocchi unici rischia che poi sono presenti delle informazioni non necessarie per le domande che l'utente fornirà al modello;
- quando il modello riceve in ingresso una quantità enorme di informazione esiste il rischio che la capacita di trovare ed elaborare i dettagli cala di molto;
- questa tecnologia permette di rendere più efficiente l'utilizzo dei token e rendere il modello più preciso nelle risposte.

2.4.2 Fact-based RAG

Il Fact-based RAG, come descritto da Chhikara et al. [5], è una architettura di memoria in cui tale sistema analizza l'ingresso fornito dall'utente per ottenere fatti dettagliati. Il funzionamento dell'architettura è il seguente:

- quando viene fornita una informazione in ingresso il modello linguistico analizza la discussione per ottenere i fatti candidati;
- dopo che il modello ha estratto i fatti, viene effettuato un confronto con la memoria esistente e i fatti estratti per avere i fatti più simili di significato attraverso i vettori di significato.

I principali vantaggi nell'utilizzare l'architettura Fact-based RAG secondo Chhikara et al. [5] sono i seguenti:

- utilizzando i fatti sintetici permette di ridurre molto l'utilizzo dei token;
- attraverso tale tecnologie si può ricordare le informazioni, in modo dettagliato, dell'utente attraverso delle sessioni a distanza;

- permette di risolvere il problema principale della perdita di contesto quando le conversazioni diventano molto lunghe.

2.4.3 Gestione dinamica della memoria

La gestione dinamica della memoria è una architettura di memoria in cui le informazioni vengono elaborate continuamente nel tempo attraverso operazioni di estrazione, consolidamento, aggiornamento e recupero delle informazioni importanti secondo Chhikara et al [5].

Il funzionamento di tale meccanismo è formato dai seguenti passaggi:

1. il sistema elabora la discussione tra l'utente e il modello ricevuta in ingresso per estrarre i fatti importanti contenuti nella discussione;

2. i fatti estratti dal sistema vengono confrontati con l'archivio di documenti per scegliere una operazione tra le quattro: ADD, UPDATE, DELETE e NOOP;

3. dopo che il sistema ha scelto l'operazione, la memoria viene aggiornata in modo tale che sia pronta per le prossime domande dell'utente.

L'operazione ADD permette di aggiungere, se non era presente, una nuova informazione nella memoria, l'operazione UPDATE permette di aggiornare l'informazione vecchia con i nuovi dati trovati, l'operazione DELETE permette di cancellare un informazione che non è più necessaria e invece l'operazione NOOP viene scelta quando il sistema non vuole eseguire nessuna operazione.

Questa architettura di memoria secondo Chhikara et al [5] permette quindi ridurre alcuni dei seguenti problemi: l'aggiornamento delle informazioni dato che cambiano nel tempo, conservare tutte le informazioni anche quelle non necessarie non è una buona idea dato che permette di avere più risposte errate, permette di controllare se ci sono dei dati duplicati o dati errati.

2.4.4 Memoria basata su grafi

La memoria basata su grafi, descritta nei lavori di Chhikara et al. e Xu et al. [5, 6], è una architettura di memoria che organizza le informazioni in una struttura dati a grafo orientato. Il grafo orientato, secondo Chhikara et al. [5], è strutturato in questo modo:

- i nodi del grafo rappresentano le entità delle conversazioni tra l'utente e il modello;
- gli archi del grafo rappresentano le relazioni con le entità;
- le etichette del grafo definiscono le categorie in cui appartengono le entità e le relazioni.

Dato che l'utilizzo delle basi di dati a grafi tradizionali per la memoria degli agenti ha il limite di dipendere da degli schemi predefiniti e di conseguenza l'agente fa più fatica ad evolvere la struttura della propria conoscenza, i ricordi vengono organizzati in una rete dinamica di note come descritto da Xu et al. [6]. Quindi invece di usare i dati in una struttura rigida, il modello linguistico studia le note e poi trova le relazioni per generare dei nuovi collegamenti flessibili per permettere di aggiornare la memoria aggiungendo, togliendo o modificando la rete dinamica.

Grazie a tale struttura di dati l'agente riesce a ricostruire sequenze di ragionamenti attraverso gli archi del grafo avendo quindi prestazioni superiori rispetto alle basi di dati a grafi tradizionali, vengono ridotti i costi computazionali per le operazioni di memoria e il tempo di recupero rimane contenuto.

2.4.5 Memoria gerarchica

La memoria gerarchica, descritto da Packer et al. [7], è una architettura di memoria a strati in cui è ispirata direttamente dalla memoria virtuale dei sistemi operativi. Questa architettura [7] separa la memoria in due parti: la memoria di contesto che è simile alla memoria RAM e il contesto esterno che è simile alla memoria secondaria.

Questi due livelli vengono gestiti dal modello usando le chiamate di funzione che trovano come analogia alle chiamate di sistema dei sistemi operativi.

Nei dettagli il funzionamento è il seguente:

- nella memoria di contesto rappresenta la finestra di contesto in cui il modello decide che operazioni eseguire nella memoria avendo a disposizione le istruzioni di sistema, uno spazio a disposizione su cui tenere le informazioni importanti da ricordare sempre e una lista delle ultime interazioni tra l'utente e il modello;
- nel contesto esterno rappresenta tutte le informazioni che il modello ha archiviato in cui possono essere ad esempio una serie di file di testo lunghi.
- quando la lista delle ultime interazioni sta per terminare lo spazio disponibile, il sistema avvisa il modello così che il modello aggiorna il contesto esterno con le informazioni più vecchie della lista delle ultime interazioni. Quando invece il modello ha bisogno di informazioni passate, effettua una ricerca nel contesto esterno per poi avere le informazioni passate nella memoria di contesto.

Questa architettura secondo Packer et al. [7] permette di risolvere i seguenti tre problemi:

- permette di superare i limiti della finestra di contesto dato che ha una capacità limitata di conseguenza con questi limiti i modelli linguistici non riescono a sostenere lunghe conversazioni con l'utente per molto tempo;

- permette di evitare di ingrandire la finestra di contesto che ha un costo computazionale più alto;
- permette di evitare che il modello perda l'attenzione al centro del testo fornito in ingresso dall'utente.

2.5 Valutazione di un sistema di memoria e metriche

La valutazione di un sistema di memoria [2, 1, 3, 4, 5, 6, 7] è un insieme di metodi in cui servono per valutare quanto il modello riesca a mantenere la coerenza e la conoscenza nel tempo in conversazioni lunghe con l'utente quindi non esaminando la singola risposta generata dal modello.

le modalità di valutazione sono le seguenti:

- la valutazione diretta analizza da sola la componente di memoria. Può essere oggettiva oppure soggettiva. La valutazione diretta oggettiva misura in modo numerico le risposte usando le metriche come ad esempio l'accuratezza. La valutazione diretta soggettiva invece viene effettuata da esaminatori umani che valutano la qualità delle risposte;

- la valutazione indiretta misura l'utilità della memoria usando il completamento di alcuni compiti come ad esempio discussioni tra più sessioni in cui se avranno successo è merito del giusto funzionamento della memoria.

La valutazione funziona in questo modo dopo aver fornito l'intero storico delle discussioni tra l'utente e il modello:

1. si osserva se l'assistente ricorda già l'informazione;

2. viene verificata la capacità di collegare, analizzare e rendere brevi le informazioni presenti nelle sessioni;

3. si osserva se il sistema riconosce che delle informazioni sono state aggiornate nel tempo;

4. viene analizzato il ragionamento temporale dell'ordine in cui sono arrivati in ingresso i messaggi;

5. si osserva se il sistema si astiene dal rispondere ad una domanda se non conosce la risposta.

Dopo aver definito le metodologie di valutazioni, bisogna introdurre le metriche [1, 4, 5, 6] utilizzate per effettuare tali metodologie di valutazioni.

2.5.1 Recall@k

Questa metrica, secondo Wu et al. [4], misura la percentuale di quante informazioni sono state recuperate:

$$\text{Recall@}k(q) = \frac{|R_k \cap D_{\text{rel}}|}{|D_{\text{rel}}|} \tag{2.1}$$

2.5.2 Exact Match

Questa metrica, secondo Minaee et al. [2], misura la percentuale di quante risposte generate da un modello sono uguali rispetto a quelle attese:

$$\text{EM} = \frac{M}{N} \tag{2.2}$$

2.5.3 Correct Abstention Rate

La quota di astensioni corrette sulle domande non rispondibili viene indicata come Correct Abstention Rate (CAR). La sua definizione è equivalente alla *Unanswerable Accuracy* (UAC) proposta da Madhusudhan et al. [8]:

$$\text{CAR} = \frac{\text{TN}}{|U|} \tag{2.3}$$

Capitolo 3

Stato dell'arte

Attraverso il capitolo precedente 2, è stato possibile definire le varie tecnologie che sono state utilizzate per la presente tesi. Questo capitolo ha l'obiettivo di definire quali concetti sono stati applicati e da quali lavori sono stati selezioni e con quali aspetti verranno confrontati in modo tale che condurranno ai principi utilizzati per la presente tesi.

3.1 Tecniche di gestione del contesto

Le tecniche di gestione del contesto, come descritte da Mei et al. [9], servono per gestire in modo efficiente la memoria dei modelli linguistici. Il contesto viene trattato non come semplice testo ma come un insieme di informazioni per migliorare la generazione delle risposte del modello linguistico. Queste tecniche affrontano le seguente aree: limiti della finestra di contesto, utilizzo di architetture di memoria per mantenere le informazioni nel lungo tempo, utilizzo di tecniche per la compressione del testo per rendere i modelli più efficienti.

3.1.1 Cronologia completa

La cronologia completa, come descritta in [4, 10], è una tecnica di gestione del modello in cui consiste nel passare in ingresso tutto lo storico delle conversazioni tra il modello linguistico e l'utente nel messaggio che si desidera mandare al modello senza utilizzare sistemi esterni di memoria come ad esempio le architetture di memoria.

Nel progetto *LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory* [4] affronta il problema in cui per fare accedere al modello il testo grezzo attraverso la memoria completa, quest'ultima riassuma o indicizza la cronologia prima di passarla al modello perdendo così informazioni.

La cronologia completa, nel progetto [4], è stata valutata utilizzando la modalità di lettura a posteriori della cronologia in cui la valutazione è stata effettuata su un insieme di dati e utilizzando l'impostazione di condizione di riferimento con evidenze note per fornire al modello soltanto le informazioni necessarie.

Lo studio di Wu et al. [4] ha scoperto le seguenti cose:

- la architettura è semplice da gestire dato che non richiede progettazione e manutenzioni complicate;
- quando le informazioni contengono evidenze importanti, i modelli riescono a raggiungere prestazioni elevate;
- quando l'intero storico delle conversazioni viene passato in ingresso al modello, avviene un calo delle prestazioni a differenza della modalità di lettura a posteriori della cronologia;
- passare l'intero storico delle conversazioni ha un enorme costo computazionale;

Il lavoro di Wu et al. [4] è importante per la presente tesi perché descrive la cronologia completa come fasi di lettura della memoria, recupero e della generazione dei indici senza passare direttamente ingresso l'insieme di dati.

Nel progetto *Evaluating Very Long-Term Conversational Memory of LLM Agents* [10] affronta il problema in cui le conversazioni molto lunghe con finestre di contesto piccole vengono troncate. Le sessioni vecchie vengono eliminate per fare spazio alle nuove sessioni.

La cronologia completa, nel lavoro di Maharana et al. [10], è stata valutata utilizzando:

- diverse configurazioni della finestra di contesto i modelli utilizzati;
- utilizzando i RAG per estrarre le informazioni in una base di dati.

I risultati del lavoro di Maharana et al. [10] sono i seguenti:

- più diventa grande la finestra di contesto e più aumenta l'accuratezza nelle risposte superando le prestazioni dei modelli che avevano finestre di contesto più piccole;
- permette di evitare a trasformare le conversazioni in riassunti;
- più testo i modelli ricevono in ingresso e più i modelli possono cadere nelle domande a trabocchetto;
- i modelli che hanno una finestra di contesto più grande possono avere degli errori sulla elaborazione temporale delle conversazioni;
- i modelli che hanno una finestra di contesto più grande non hanno le stesse capacità di ricordare come nella mente umana.

Il lavoro di Maharana et al. [10] è importante per la presente tesi perché ha fornito delle idee di confronto per le diverse configurazioni di memoria conversazionale e come valutare la memoria in termini di prestazioni.

3.2 Sistemi di memoria conversazionali esistenti

Questa parte si occupa di spiegare come dei sistemi di memoria conversazionale gestiscono alcuni problemi per la gestione della memoria rappresentando le informazioni, come le gestiscono, come le recuperano e quali vantaggi e limiti hanno avuto. Questi sistemi di memoria conversazionali sono importanti per la presente tesi perché sono i principali riferimenti per la progettazione dei loro adattamenti semplificati.

3.2.1 MemGPT

MemGPT, proposto da Packer et al. [7], è un sistema di memoria per estendere e gestire la memoria degli LLM ispirandosi alla memoria virtuale dei sistemi operativi. Invece di usare soltanto la finestra di contesto, MemGPT utilizza come principio la paginazione della memoria virtuale in cui:

- la finestra di contesto viene considerata come la memoria RAM;
- le informazioni storiche vengono conservate in una memoria esterna, analoga alla memoria secondaria;
- le chiamate di funzione che esegue il modello sono ispirate alle chiamate di sistema dei sistemi operativi.

MemGPT [7] ha l'obiettivo di affrontare il problema della finestra di contesto dato che nelle conversazioni lunghe i modelli linguistici hanno dei problemi precedentemente descritti.

Gli autori [7] hanno valutato MemGPT in due modi raggiungendo i seguenti risultati:

- attraverso l'uso degli agenti conversazionali basati su MemGPT in cui venivano testati sui ricordi di fatti dettagliati di conversazioni oppure veniva valutata la capacità di

iniziare nuove conversazioni con l'utente attraverso l'uso della memoria. MemGPT ha avuto un incremento enorme dell'accuratezza e nell'altro esperimento ha raggiunto punteggi simili ai messaggi scritti dagli umani.

- attraverso l'analisi dei documenti complessi su grandi articoli in cui MemGPT ha mantenuto alte prestazioni oppure un verifica di ricerca di tanti passaggi su dati in sequenza in cui MemGPT ha recuperato le informazioni correttamente in più livelli.

Gli autori di MemGPT [7] hanno mostrato i seguenti vantaggi e limiti:

- MemGPT riesce a gestire i documenti estesi senza che l'architettura di memoria del modello venga modificata;
- MemGPT riesce a mantenere la coerenza nella memoria a lungo termine;
- attraverso le chiamate di funzione, l'agente riesce ad ottenere le informazioni nella memoria esterna;
- MemGPT dipende molto dalla capacità del modello base su cui viene costruita sopra l'architettura di memoria;
- è stato mostrato che MemGPT tende ad interrompere prima la ricerca delle informazioni nell'analisi documentale;
- le chiamate di funzione comportano più costi computazionali.

Il lavoro di Packer et al. [7] è importante per la presente tesi perché fornisce il riferimento da cui è stato preso il principio della memoria gerarchica ottenendo quindi adattamento semplificato e controllato e non una replica di MemGPT.

3.2.2 Mem0 e la sua estensione a grafo Mem0g

Chhikara et al. [5] propongono Mem0 che è una architettura di memoria in cui è stata creata per fornire supporto agli agenti una memoria gestibile. Mem0 ottiene soltanto le informazioni più importanti grazie ai seguenti passaggi:

- in ingresso vengono ricevute le informazioni dei messaggi tra dell'utente e del modello, il modello analizza tali informazioni insieme all'archivio di documenti e la lista di messaggi recenti per ottenere i fatti dell'utente;
- i fatti estratti dal modello vengono poi confrontati con l'archivio di documenti in cui il modello linguistico decide quali tra le seguenti quattro operazioni eseguire: ADD (aggiungi una nuova informazione), UPDATE (aggiorna una informazione esistente), DELETE (elimina una informazione) o NOOP (nessuna modifica).

L'obiettivo degli autori attraverso Mem0 [5] è quello di risolvere i limiti della finestra di contesto dei modelli linguistici durante le lunghe discussioni con l'utente.

Gli autori di Mem0 [5] hanno valutato l'architettura di memoria attraverso dei dialoghi estesi di diverse categorie ed hanno utilizzato le metriche per valutare le prestazioni del modello analizzando oltre a quelle anche il suo consumo dei token.

Gli autori di Mem0 [5] hanno trovati i seguenti vantaggi e limiti:

- questa architettura ha una elevata accuratezza superando molti sistemi di memoria;
- è molto efficiente nell'utilizzo dei token riducendone i costi;
- è molto veloce ad effettuare le ricerche delle informazioni;
- la memoria basata su testo ha qualche limite quando deve ottenere le relazioni rigide tra le entità

Chhikara et al. [5] hanno proposto una variante di Mem0 basata sui grafi orientati chiamata: Mem0g.

Mem0g [5] memorizza le informazioni sottoforma di grafo orientato dove i nodi rappresentano le entità e gli archi invece le relazioni tra le entità.

Mem0g [5] risulta molto bravo nelle domande temporali grazie alla sua struttura a grafo che riesce a ricostruire bene la sequenza cronologica dei messaggi.

L'unica nota negativa di questa architettura [5] è che richiede più utilizzo dei token.

Le architetture di Chhikara et al. [5] sono importanti per la presente tesi perché mostrano come gestire la memoria conversazionale con fatti sintetici, come aggiornale attraverso delle apposite operazioni ed usare le strutture entità e relazione. Questi principi hanno aiutato a definire le idee delle configurazioni da utilizzare per la presente tesi che sono degli adattamenti semplificati e controllati e non delle repliche fedeli.

3.2.3 A-MEM

Xu et al. [6] propongono A-MEM che è un sistema di memoria agentica che organizza le informazioni archiviate in modo dinamico ed autonomo attraverso l'uso di agenti.

A-MEM [6] è composto da quattro parti importanti:

- quando avviene una nuova discussione tra l'utente e il modello, il sistema genera una descrizione di tale discussione;
- il sistema trova le informazioni storiche ed utilizza il modello linguistico per effettuare i collegamenti;
- sulla base delle nuove informazioni, il sistema può aggiornare le informazioni archiviate;
- per generare una risposta l'agente confronta la domanda che gli è stata fornita in ingresso con le note che possiede.

Gli autori di A-MEM [6] hanno affrontato il problema delle strutture di memorizzazione fissate che sono molto rigide ed hanno una capacità limitata di generalizzazione.

Gli autori di A-MEM [6] hanno valutato tale sistema in questo modo: mantenendo fissi l'insieme dei dati e i modelli, confrontato con altre architetture di gestione della memoria, mantenendo fisse le metriche di valutazione.

Dalla ricerca che hanno effettuato gli autori di A-MEM [6] sono emersi i seguenti vantaggi e limiti:

- è superiore nei compiti complessi avendo dei punteggi di accuratezza più alti;
- è più efficiente nell'utilizzo dei token;

- ha una complessità spaziale lineare senza informazioni in più;
- la rete riesce ad evolversi da sola senza avere degli schemi fissi;
- dipende dal modello linguistico utilizzato alla base;
- è specializzato soltanto nel testo;
- bisogna impostare un numero moderato di note per ottenere il meglio delle informazioni senza che vengano passate al modello delle informazioni inutili.

Il lavoro di Xu et al. [6] è importante per la presente tesi perché mostra come la memoria può essere rappresentata in una struttura flessibile e molto connessa. Questo principio [6] insieme a quello di Mem0 e Mem0g [5] ha aiutato alla creazione di una configurazione semplificata per la presente tesi.

3.3 Sintesi critica e posizionamento del presente lavoro

Gli approcci descritti hanno tutti in comune l'obiettivo di superare i limiti della finestra di contesto per conservare le informazioni passate e e di recuperare tali informazioni quando l'utente effettua delle domande al modello linguistico.

Principalmente gli approcci descritti si possono differire in breve in:

- la cronologia completa, come descritta in [4, 10], conserva tutto lo storico;
- MemGPT di Packer et al. [7] utilizza la memoria gerarchica;
- Mem0 di Chhikara et al. [5] si basa sui fatti dell'utente e li gestisce con delle apposite operazioni
- Mem0g di Chhikara et al. [5] utilizza i grafi orientati per gestire in modo flessibile le informazioni;
- A-MEM di Xu et al. [6] utilizza delle note collegate per descrivere in breve le informazioni;

Il compromesso che è stato osservato dagli approcci descritti è:

- per ottenere più informazioni ci saranno più costi;
- per ottenere più sintesi ci saranno più perdita di informazioni;
- più le strutture sono complesse e più la gestione delle informazioni è complicata;
- più il recupero è selettivo e più aumenta il rischio di recuperare le informazioni non necessarie.

La presente tesi si colloca nel confronto controllato di diverse rappresentazioni di memoria nell'utilizzo di architetture semplificate prendono come fonte di ispirazioni i sistemi esistenti in cui non sono una replica completa bensì l'obiettivo è quello di studiare come le informazioni vengono memorizzate, recuperate ed utilizzate per rispondere alle domande dell'utente.

Capitolo 4

Framework sperimentale

Attraverso il capitolo precedente 3, sono stati esplorati i lavori che hanno aiutato la creazione del presente framework come fonte di ispirazione. Questo capitolo ha l'obiettivo di spiegare come è strutturato il presente framework e in che modo hanno influenzato i lavori che sono stati analizzati e in quali problematiche verrà utilizzato.

4.1 Obiettivo

Questa parte ha l'obiettivo di descrivere quali problemi vengono affrontati dal framework sperimentale, quali sono gli obiettivi del presente framework e cosa deve osservare.

4.1.1 Problemi

Uno dei problemi che vengono analizzati nella presente tesi è il fatto che le conversazioni dei modelli linguistici sono separate causando delle discontinuità tra le varie sessioni facendo così perdere delle informazioni al modello linguistico e costringendo ogni volta all'utente di rimandare quelle informazioni nei suoi messaggi.

Un altro problema è che se l'utente invia una grande quantità di informazioni al modello linguistico, il modello linguistico non riesce a tenere conto di tutto il contesto causando spesso delle risposte incomplete a ciò l'utente desiderava ricevere e facendogli spendere più token.

4.1.2 Finalità

L'obiettivo del framework è quello di osservare il cambiamento della memoria nei seguenti tre casi:

- attraverso l'uso della memoria esterna in cui raccogliendo le informazioni in un archivio di documenti;
- mantenere la continuità tra le varie conversazioni tra l'utente e il modello linguistico;
- analizzare il recupero selettivo delle informazioni.

4.1.3 Che cosa osservare

Le caratteristiche che vengono osservate nel presente framework sono le seguenti:

- la presenza delle informazioni che possono rispondere alla domanda fornita dall'utente in ingresso al modello linguistico;
- verificare se il recupero delle informazioni ha successo;
- osservare se le informazioni recuperate e che vengono utilizzate per generare la risposta alla domanda sono esatte.

Il framework non ha l'obiettivo di modificare internamente modelli linguistici ma dare a loro del supporto per migliorare la realizzazione delle risposte senza andare a cercare quale modello è il migliore.

4.2 Funzionamento generale



Figura 4.1: Come è strutturato il progetto

Il framework viene utilizzato dallo sperimentatore (per semplicità verrà chiamato da questo momento in poi utente) in cui per usare tale framework deve prima accedere dal terminale ed entrare nella cartella in cui è presente il framework. Come si può vedere in figura 4.1 fa da tramite tra l'utente e il modello. Dopo che l'utente è entrato nella cartella in cui è presente il framework, deve utilizzare dei comandi per decidere la configurazione da utilizzare. La configurazione è la seguente:

- il modello linguistico da utilizzare;
- la forma di memoria desiderata;
- i contenuti che il framework deve leggere come i messaggi degli scenari, l'ordine delle sessioni e le domande da sottoporre al modello;
- i parametri del processo che possono essere la presenza del numero di token da utilizzare che può essere limitata o illimitata e la quantità massima di informazioni.

Il ruolo del framework quindi è quello di leggere i messaggi predisposti, gestire la memoria, recuperare le informazioni, preparare le informazioni da passare al modello, interrogare il modello.

Quindi in uscita, l'utente vedrà la risposta del modello, le informazioni recuperati e quali invece sono state salvate.

Per utilizzare il framework bisogna eseguire i seguenti passaggi:

1. l'utente accede nel terminale nella cartella in cui è presente il framework;
2. l'utente effettua una configurazione in cui deve decidere quale modello linguistico utilizzare e la sua forma di memoria;
3. da quel momento l'utente può iniziare le conversazioni con il modello scelto.

4.2.1 Memorizzazione e gestione delle informazioni

Prima che l'utente effettua una domanda al framework, il framework deve prima leggere tutti i messaggi che sono presenti in ordine di sessione in cui tali informazioni vengono elaborate una dopo l'altra dando principale attenzione ai messaggi dell'utente.

Dopo di che il framework deve decidere quali informazioni devono venire conservate in cui può decidere anche di mantenere il messaggio originale oppure decidere di trasformare il messaggio attraverso una delle forme di memoria.

Infine il framework deve confrontare le nuove informazioni con quelle vecchie così che poi il framework dopo tale confronto aggiorna le informazioni che devono essere aggiornate perché sono superate.

4.2.2 Recupero e costruzione del contesto

Dopo che il framework riceve in ingresso la domanda dell'utente, utilizza la domanda che gli è stata fornita come richiesta di ricerca in cui gli servirà per consultare la memoria esterna e trovare un insieme di informazioni simili. Il framework confronta la domanda con le informazioni presenti nella memoria in cui per ogni informazione viene dato un livello di utilità per la domanda. Dopo di che ogni informazione viene ordinata sulla base del livello di utilità per la domanda e vengono scelte le informazioni con il livello più alto di utilità in cui il framework cerca di essere attento anche all'eventuale limite di spazio. Infine le informazioni che il framework ha selezionato saranno usate per preparare il messaggio che verrà passato al modello.

4.2.3 Generazione e tracciabilità della risposta

Il framework ha delle regole per costruire, utilizzando le informazioni che recupera, ogni tipo di messaggio. Quindi i passaggi che il framework deve eseguire affinché riesca a costruire il messaggio che verrà poi inviato al modello sono:

1. il framework legge le regole che sono: usare solo informazioni che possiede, non deve inventarsi niente che non sia supportato, se non ha informazioni a sufficienza deve dichiararlo, deve seguire la struttura predefinita;

2. il framework prende la domanda dell'utente e ci mette insieme le informazioni sulla base dei criteri che possiede organizzando il messaggio che riceverà il modello in diverse parti come le regole, le informazioni e la domanda originale dell'utente. Una cosa molto importante è che il modello non riceve anche i criteri con cui poi saranno valutate le sue risposte;

3. il modello linguistico riceve il messaggio del framework e genera la risposta. La risposta generata dal modello viene passata al framework;

4. il framework dopo aver ricevuto la risposta memorizza la domanda che gli era stata fornita dall'utente, il modello che è stato utilizzato, la risposta generata dal modello stesso e i possibili errori che ci sono stati durante l'esecuzione.

4.3 Forme di memoria considerate

I seguenti sistemi di memoria che vengono utilizzati dal framework sono degli adattamenti semplificati dei lavori originali.

4.3.1 Turn-level RAG

Nel presente progetto, è stata utilizzata una versione semplificata dell'architettura di memoria Turn-level RAG in cui si occupa del confronto delle parole senza utilizzare particolari tecnologie. Questa architettura prende ispirazione dal concetto di RAG di Lewis et al. [3] e la scomposizione dello storico dei messaggi trattata da Wu et al. [4].

4.3.2 Fact-based RAG

Nel presente progetto, è stata utilizzata una versione semplificata dell'architettura di memoria Fact-based RAG in cui è stata semplificata in un insieme di fatti strutturati. Questa architettura prende ispirazione dai lavori di:

- Wu et al. [4] per l'uso dei fatti per rendere più facile la creazione dei indici e il recupero delle informazioni;
- Chhikara et al. [5] prende ispirazione nel rendere le informazioni delle conversazioni in parti di memoria compatti.

4.3.3 Memoria con aggiornamenti

Nella presente tesi, è stata utilizzata una versione semplificata della memoria con aggiornamenti in cui l'idea deriva dal lavoro di Chhikara et al. [5] in cui sono presenti delle regole esplicite per distinguere se le informazioni sono correnti, vecchie oppure non sono più presenti. Questa architettura di memoria viene utilizzata in uno degli esperimenti come estensione dell'architettura Fact-based RAG.

4.3.4 Memoria a grafo

Nella presente tesi, è stata utilizzata un'adattamento semplificato della memoria a grafo in cui l'idea deriva dai lavori di Chhikara et al. [5] e di Xu et al. [6] in cui non utilizza un sistema totalmente a grafo ma una rappresentazione semplice con recupero relazionale.

4.3.5 Memoria gerarchica

Nel presente progetto, è stata usato un'adattamento semplificato della memoria gerarchica in cui l'idea deriva dal lavoro di Packer et al. [7] in cui l'ispirazione la separazione tra il contesto principale e memoria esterna come proposto dal suo lavoro [7] ma utilizza regole per il recupero delle informazioni e collocazione senza che viene affidata la gestione al modello.

4.3.6 Cronologia completa

La cronologia completa, come descritta in [4, 10], è una tecnica di gestione del contesto quindi non è una architettura di memoria. Nella presente tesi viene utilizzata come controllo diagnostico per confrontare il comportamento del modello quando ha accesso a tutte le informazioni.

Capitolo 5

Progettazione degli esperimenti

Attraverso il capitolo precedente 4 è stato possibile definire la struttura del framework sperimentale con cui viene utilizzato per eseguire gli esperimenti. Il seguente capitolo ha l'obiettivo di definire le domande di ricerca, come sono stati impostati gli esperimenti e con quali criteri vengono valutati. Come dominio applicativo per gli scenari sperimentali è stato scelto quello della sicurezza informatica utilizzando come insieme di dati dei file JSON in cui sono delle sequenze di messaggi tra l'utente e l'assistente suddivise in sessioni. Quindi il contenuto dell'insieme di dati è il dialogo.

5.1 RQ1 — Effetto della disponibilità della memoria

L'obiettivo di questa domanda di ricerca è quello di osservare come cambiano le risposte del modello in base alle possibili configurazioni di memoria mantenendo fisse alcune condizioni

5.1.1 Disegno dell'esperimento

Sono stati utilizzati due scenari di sicurezza informatica, questi scenari sono fittizi e sono stati costruiti proprio per l'esperimento e non derivano da dati reali.

Il primo scenario (chiamato anche scenario numero 1) simula la gestione di un incidente informatico che è composto in quattro sessioni in questi passi:

1. la prima sessione indica che è stato rilevato un accesso anomalo tramite un account e quindi la prima ipotesi è che la password è stata violata;

2. la seconda sessione indica che i registri degli accessi smentiscono l'ipotesi iniziale e quindi viene scoperto che la causa era stata un token API che quindi deve essere sostituito;

3. la terza sessione dice che il token API è stato sostituito con un nuovo token con i privilegi ridotti completando così il contenimento. Restano tuttavia due verifiche da effettuare;

4. Nella ultima sessione è stata introdotta una scadenza del rapporto di tale incidente e che le due verificare da fare hanno la priorità.

Il secondo scenario (chiamato anche scenario numero 2) simula un gruppo di lavoro che cerca di correggere un problema del recupero di password per un sito di commercio. Questo scenario è formato dai seguenti quattro scenari:

1. nella prima sessione, tre profili hanno subito un cambio di password non autorizzato, quindi viene deciso di introdurre una verifica di CAPTCHA;

2. nella seconda sessione, dai registri delle operazioni di recupero della password si scopre che ogni profilo ha una sola richiesta ma il collegamento è stato utilizzato due volte allora l'idea del CAPTCHA è stata abbandonata. Viene deciso quindi di usare una regola di usare token utilizzabili per 15 minuti in cui scaduto quei 15 minuti il token non è più utilizzabile;

3. nella terza sessione, la soluzione proposta funziona però deve essere verificata il funzionamento dell'applicazione e di comportamento del messaggio su posta elettronica arrivato in ritardo;

4. nell'ultima sessione, la verifica dell'applicazione ha successo. Rimane quindi solo lo studio del messaggio di posta elettronica arrivato tardi in cui il risultato deve spiegato in una riunione programmata.

5.1.2 Configurazione sperimentale

Per l'esperimento della domanda di ricerca è stato utilizzato il seguente modello linguistico: Claude Sonnet 5 di Anthropic [11] in cui si è occupato di generare le risposte sperimentali. Le chiamate al modello sono state effettuate utilizzando Claude Code CLI, impostando il livello di ragionamento su medium [11]. Per l'esperimento domanda che viene fornita nell'esperimento viene eseguita da sola e le risposte precedenti non diventano informazioni per le domande successive.

Per questo esperimento sono state decise delle configurazioni di memoria che influenzano solo quali messaggi precedenti Claude può visualizzare.

Le configurazioni di memoria sono:

- il modello riceve solo la domanda quindi non conosce le sessioni precedenti. Questa configurazione si chiama memoria assente;

- il modello riceve solo le informazioni della sessione corrente ossia la numero quattro. Questa configurazione si chiama memoria locale;

- il modello riceve tutte le informazioni delle varie sessioni. Questa configurazione si chiama memoria condivisa.

Per il sistema di recupero delle informazioni è stata utilizzata l'architettura di memoria Turn-level RAG in cui è stato impostato per recuperare due messaggi per domanda ricevuta in ingresso.

5.1.3 Valutazione

Per la valutazione sono state definite sette domande per scenario, quindi in totale 14 domande in questo esperimento, che sono: obiettivi, decisioni aggiornate e informazioni datate, attività completata, attività non svolta, informazione presente nella sessione corrente, come sono state collegate le diverse sessioni, che informazioni non sono mai state fornite.

L'obiettivo della valutazione è quello di osservare se l'informazione è presente nella memoria, come valutazione si analizza se l'informazione è disponibile e ciò viene scoperto sulla base di cosa dice il modello in cui se il modello risponde allora viene conteggiata altrimenti no. Dopo di che see l'informazione è stata recuperata e per fare ciò si utilizza la metrica recall@k in cui è strutturata in questo modo:

$$\text{Valore recuperato} = \begin{cases} 1 & \text{se Recall@k} = 1 \\ 0 & \text{altrimenti} \end{cases} \tag{5.1}$$

Nel caso in cui il valore recuperato è pari ad uno allora viene conteggiato. Questo modo di utilizzo della presente metrica è stato ispirato dal lavoro di Wu et al. [4].

Oltre alla metrica Recall@k viene utilizzata le metriche Exact match per misurare se la risposta è corretta quando il valore è 1, Correct abstention rate quando l'astensione è correte quando il valore è 1.

Per capire se i risultati dell'esperimento sono sensati, è stato eseguità una prova preliminare manuale in cui sono stati inviati manualmente dei messaggi a Claude nelle

stesse condizioni in cui è stato fissato l'esperimento. Nel complessivo ci sono state 42 prove nei due scenari di cui 39 su 42 comportamenti appropriati.

Per questo esperimento come ulteriore valutazione viene utilizzato la cronologia completa come controllo diagnostico dando a Claude tutte le informazioni necessarie.

5.2 RQ2 — Effetto della rappresentazione della memoria

L'obiettivo di questa domanda di ricerca è quello di osservare quale strategia di memorizzazione è la migliore utilizzando le architetture di memoria nel confronto usando gli stessi scenari e lo stesso modello.

5.2.1 Disegno dell'esperimento

Sono stati utilizzati quattro scenari di sicurezza informatica, questi scenari sono fittizi e sono stati costruiti proprio per l'esperimento e non derivano da dati reali.

Il primo scenario che è stato utilizzato è lo scenario numero 2 della domanda di ricerca già usato nella prima domanda di ricerca.

Il secondo scenario (chiamato anche scenario numero 3) racconta di un incidente informatico in cui riguarda un computer dato che sono presenti dei segnali sospetti come ad esempio che il computer è lento, un file con una richiesta di pagamento e un processo ignoto.

All'inizio era stato ipotizzato che era un programma che cifra i documenti per poi chiederne un riscatto ma poi viene escluso perché era di una simulazione vecchia. Lo studio ha mostrato che sono state rubate le credenziali e sono stati inviati dei dati e quindi il programma è stato strutturato per rubare i dati. Il computer viene isolato e le

informazioni che erano state rubate vengono restituite tuttavia però l'origine della infezione è sconosciuta.

Il terzo scenario (chiamato anche scenario numero 4) riguarda un caso di Smishing in cui un lavoratore riceve un messaggio dal suo numero di telefono per la consegna di un pacco tuttavia ha aperto invece una pagina in cui ha inserito le proprie credenziali e quindi poco dopo avviene un accesso anomalo attraverso il suo profilo aziendale. I truffatori con il profilo della vittima cercano di ottenere tutti i messaggi della posta elettronica con la parola fattura allora l'azienda scopre il caso ed ha modificato le credenziali del dipendente fermando così i truffatori. L'unica cosa che non è stato scoperto è come i truffatori hanno il numero di telefono della vittima.

Per l'ultimo scenario (chiamato anche scenario numero 5) composto da nove sessioni, riguarda un accesso anomalo nel portale di una azienda. La prima ipotesi era coinvolto il server che conteneva tale portale ma poi dopo un controllo è stato scoperto che il server realmente coinvolto era quello dei dati dei fornitori. Dopo di che è stato trovato un documento che si pensava fosse una prova ma poi è stato escluso. Attraverso le verifiche sono stati rubati i dati di 1480 fornitori come ad esempio i nomi delle aziende ecc. Attraverso dei interventi di manutenzione il portale torna operativo e rimane la verifica da fare sul server dei dati dei fornitori.

5.2.2 Configurazione sperimentale

Per il seguente esperimento, viene utilizzato di nuovo il modello linguistico Claude Sonnet 5.

In questo esperimento ogni architettura verrà singolarmente eseguita nello scenario scelto. Le configurazioni architetturali per la valutazione comparativa è la seguente in tabella 5.1:

Tabella 5.1: Configurazione sperimentale per la seconda domanda di ricerca.

Scenario	**Configurazioni confrontate**	**Limite del contesto**
2	Turn-level RAG e Fact-based RAG	Senza limite
3	Fact-based RAG e Memoria con aggiornamenti	Senza limite
4	Turn-level RAG, Memoria con aggiornamenti e Memoria a grafo	Senza limite
5	Turn-level RAG, Memoria con aggiornamenti e Memoria gerarchica	200 token
Controllo	Cronologia completa	Non applicabile

Nel caso dello scenario numero 5 è stato messo un limite di 200 token per vedere quali tra le tre architetture confrontate è quella più efficiente e quella più dispendiosa.

5.2.3 Valutazione

I risultati del secondo esperimento vengono valutati attraverso l'uso delle seguenti metriche: Recall@K, Exact Match e Correct Abstention Rate utilizzando dei grafici realizzati tramite Matplotlib 3.9.4 [12].

5.3 RQ3 — Effetto della condivisione della memoria

L'obiettivo di questa domanda di ricerca è quello di osservare come delle attività diverse nello stesso dominio riescono a generare confusione alla memoria del modello linguistico.

5.3.1 Disegno dell'esperimento

Per il terzo esperimento è stato utilizzato il seguente insieme di dati reali di GitHub Advisory Database [13] in cui è un archivio pubblico che contiene le vulnerabilità software che è mantenuto da GitHub. Per l'esperimento, i dati non sono stati utilizzati direttamente nella forma originale bensì sono stati presi soltanto alcuni fatti come il nome del pacchetto, le versioni vulnerabili, quanto erano gravi le vulnerabilità e la prima versione in cui non si presentava più il problema. Tutte queste informazioni che sono state raccolte vengono trasformate in episodi fittizi sottoforma di file JSON in cui il loro scopo è quello di ingannare il modello con delle attività diverse che vengono chiamate per semplicità sottoscenari. Il numero di domande che vengono date al modello sono 120.

5.3.2 Configurazione sperimentale

La configurazione sperimentale per il presente esperimento è la seguente: il modello che viene utilizzato è Claude Sonnet 5.

5.3.3 Valutazione

Per la valutazione viene utilizzata le metrica Recall@k utilizzata nei primi due esperimenti e per il conteggio delle informazioni contaminati è stato adattato il principio di rilevanza delle informazioni pertinenti dal lavoro di Es et al. [14]

5.4 RQ4 — Continuità dopo il cambio del modello

L'obiettivo di questa domanda di ricerca è osservare se un secondo modello rispetta i vincoli e le decisioni che sono state prese dal modello precedente.

5.4.1 Disegno dell'esperimento

Per il seguente quarto esperimento è stato riutilizzato lo scenario numero 5 del secondo esperimento.

5.4.2 Configurazione sperimentale

Per il seguente quarto esperimento sono stati utilizzati tre modelli: Claude Sonnet 5, Llama 3.2 3B di Meta [15] e Gemma 3 4B di Google [16].

Questi due modelli sono stati scelti sulla base della loro taglia e della loro memoria perché vengono eseguiti in locale utilizzando Ollama [17] nella versione 0.34.2.

La configurazione dell'esperimento è la seguente in tabella 5.2:

Tabella 5.2: Configurazione del quarto esperimento

Sessioni 1–5	**Sessioni 6–9**	**Condizione**	**Cosa avrà a disposizione il modello cambiato**
Claude Sonnet 5	Llama 3.2 3B	Separata	Soltanto le sessioni 6–9
		Condivisa	Tutte le sessioni 1–9
	Gemma 3 4B	Separata	Soltanto le sessioni 6–9
		Condivisa	Tutte le sessioni 1–9

La condizione di memoria separata indica che il modello ha accesso solo a delle sessioni dedicate, in questo caso sono le sessioni dalla numero 6 fino alla numero 9, mentre la condizione di memoria condivisa indica che il modello ha accesso a tutte le sessioni presenti nello scenario.

Quindi l'esperimento si svolge in questo modo: Claude segue le sessioni dalla numero 1 fino alla numero 5, dopo di che Claude viene cambiato in cui al suo posto ci sono Llama e Gemma in cui elaborano in autonomia le sessioni dalla numero 6 fino alla numero 9.

Come architettura di memoria viene utilizzato Turn-level RAG per l'uso di tutte le prove.

5.4.3 Valutazione

Per la valutazione per il presente esperimento è stata utilizzata la metrica Recall@k come era stata utilizzata nei precedenti esperimenti in cui viene valutata su questi fronti: informazioni recenti recuperati in cui indica le ultime sessioni, informazioni storiche recuperate in cui indica le prime sessioni, come le informazioni sono state recuperate da Llama e Gemma.

5.5 RQ5 — Effetto del modello generativo

L'obiettivo di questa domanda di ricerca è quello di osservare quanto i risultati dipendono dal modello linguistico scelto.

5.5.1 Disegno dell'esperimento

Per il seguente ultimo esperimento è stato utilizzato il seguente insieme di dati reali di VERIS Community Database (VCDB) [18, 19] in cui è una raccolta di incidenti di sicurezza informatica. Per l'esperimento è stata utilizzata la versione del [19] in cui sono stati presi dei dati di partenza dei vari incidenti e sono stati costruiti sopra delle domande sintetiche in formato JSON in cui il suo numero di domande sono 2016.

5.5.2 Configurazione sperimentale

Per la configurazione sperimentale sono stati utilizzati i seguenti due modelli: Llama 3.2 3B di Meta [15] e Gemma 3 4B di Google [16] che sono stati già usati nel quarto esperimento. Questi due modelli elaborano tutte le domande sintetiche che sono state costruite su

dati veri in autonomia avendo quindi nel totale 4032 domande elaborate. L'esperimento viene effettuato in due condizioni: contesto sufficiente in cui ci sono dati a sufficienza affinché il modello può rispondere bene e il contesto insufficiente in cui i dati non sono sufficienti per il modello. Questa impostazione sperimentale è stata presa da principio dal lavoro di Maharana et al. [10] in cui come dominio è stato utilizzato quello della sicurezza informatica.

5.5.3 Valutazione

La valutazione dell'ultimo esperimento viene effettuata attraverso le condizioni sperimentali appena descritte tramite l'uso delle seguenti due metriche: Exact Match nel caso del contesto sufficiente e Correct Abstention Rate nel caso del contesto insufficiente. Queste due metriche hanno l'obiettivo di valutare se i modelli rispondo bene sulla base della quantità di informazioni che possiedono.

Capitolo 6

Risultati sperimentali

Attraverso il capitolo precedente 5 è stato possibile definire l'impostazione sperimentale della presente tesi, questo capitolo ha l'obiettivo di illustrare i risultati degli esperimenti eseguiti.

6.1 Risultati della prima domanda di ricerca

I risultati della prima domanda di ricerca consentono di valutare se la disponibilità delle informazioni disponibili delle sessioni precedenti permette di migliorare la correttezza delle risposte.

6.1.1 Risultati osservati

Tabella 6.1: Risultati relativi alla prima domanda di ricerca

Condizione	**Informazione disponibile**	**Informazione recuperata**	**Risposta corretta**	**Astensione corretta**
senza memoria	0/14	non applicabile	non applicabile	14/14
memoria locale	5/14	5/5	5/5	8/9
memoria condivisa	12/14	11/12	11/11	2/2

Come è stato detto nello scorso capitolo, l'obiettivo di questa domanda di ricerca è quello di osservare se in assenza, parziale presenza e totale presenza di memoria la qualità delle risposte generate dal modello linguistico migliorano. La seguente tabella 6.1 illustra i risultati del primo esperimento in cui si può osservare che le risposte generate dal modello si astengono completamente alla domanda. Nel caso della memoria locale il modello riesce a recuperare le informazioni ed a rispondere alle domande possiede o meno le informazioni per la domanda però si astiene quasi completamente osservando il numero di astensioni.

Nella memoria condivisa invece, grazie al fatto che il modello contiene tutte le informazioni, riesce a rispondere quasi sempre bene.

6.1.2 Risposta alla domanda di ricerca

Osservando i dati dei risultati si può dedurre che la risposta alla domanda di ricerca se la memoria è utile o meno è: avere a disposizione le informazioni passate permette di avere un miglioramento nelle risposte e quindi è utile avere a disposizione la memoria.

6.2 Risultati della seconda domanda di ricerca

I risultati della seconda domanda di ricerca consentono di analizzare come differenti modalità di organizzazione della memoria influenzano la qualità delle risposte e il recupero delle informazioni.

6.2.1 Risultati osservati

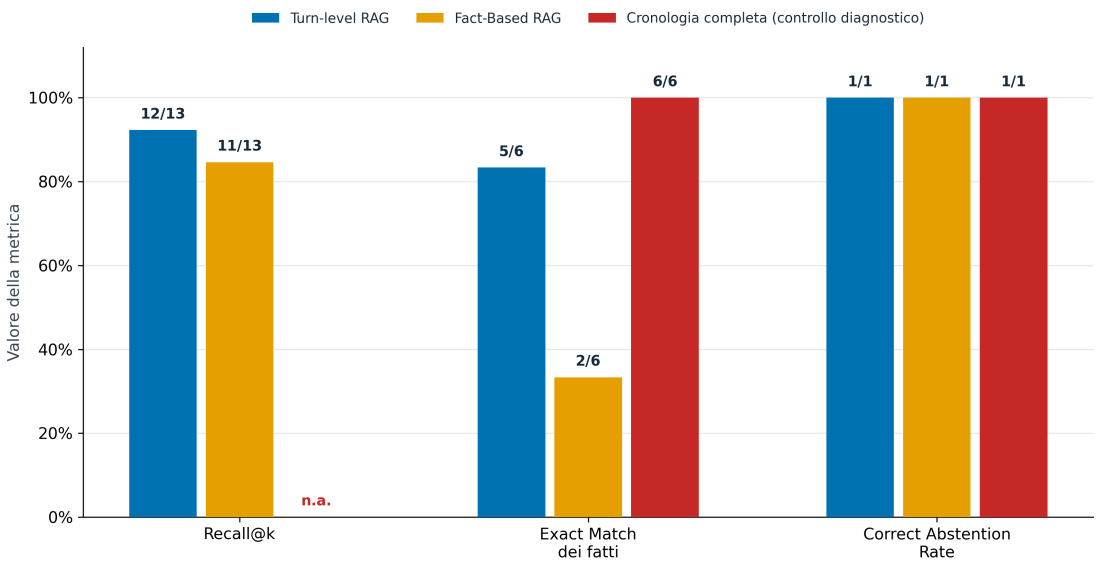


Figura 6.1: Confronto nello scenario 2

Dal seguente grafico 6.1 del primo confronto si può osservare che:

- nel recuperare le informazioni, Turn-level RAG è leggermente migliore di Fact-based RAG mentre la cronologia completa che viene usata come controllo diagnostico non può essere classificabile in questo confronto perché ha accesso all'intero storico senza effettuare dei recuperi;

- nell'indicare se le risposte sono fedeli alle informazioni in possesso, Turn-level RAG supera Fact-based RAG mentre la cronologia completa è nettamente superiore ad entrambe le architetture perché ha accesso all'intero storico;

- nell'astenersi alle domande di cui non si possiedono le fonti necessarie nel risponderle, entrambe le strategie di memoria si astengono correttamente.

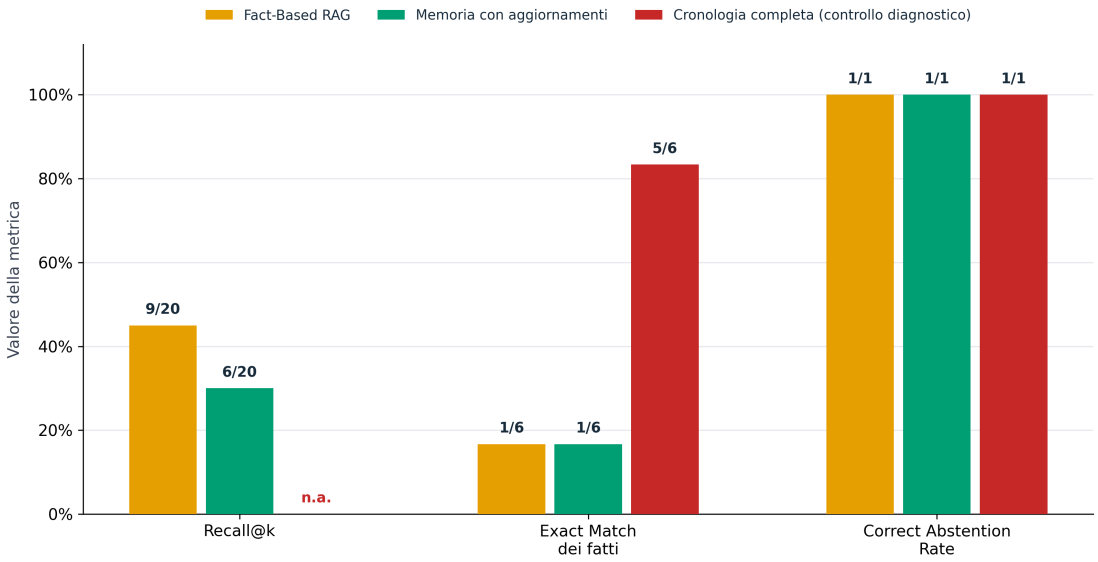


Figura 6.2: Confronto nello scenario 3

Dal seguente grafico 6.2 del secondo confronto si può osservare che:

- nel recuperare le informazioni, Fact-based RAG è supera la memoria con gli aggiornamenti mentre la cronologia completa che viene usata come controllo diagnostico non può essere classificabile in questo confronto perché ha accesso all'intero storico senza effettuare dei recuperi;

- nell'indicare se le risposte sono fedeli alle informazioni in possesso, entrambe le due architetture non hanno dei risultati ottimi perché non hanno tutte le informazioni necessarie per rispondere bene alle domande mentre la cronologia completa è nettamente superiore ad entrambe le architetture perché ha accesso all'intero storico;
- nell'astenersi alle domande di cui non si possiedono le fonti necessarie nel risponderle, entrambe le strategie di memoria si astengono correttamente.

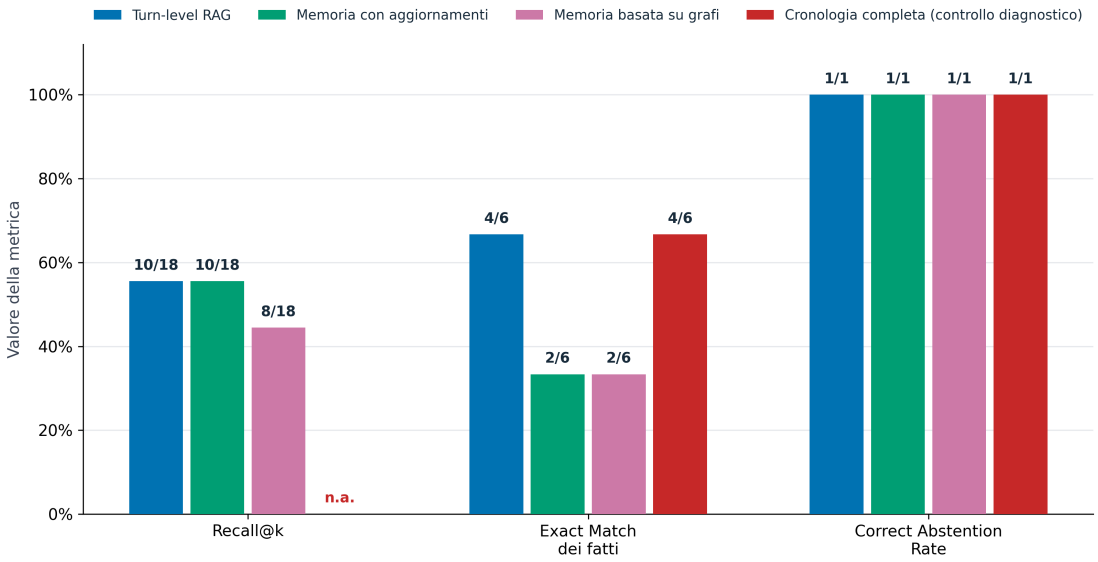


Figura 6.3: Confronto nello scenario 4

Dal seguente grafico del terzo confronto si può osservare che:

- nel recuperare le informazioni, Turn-level RAG è pari alla memoria con gli aggiornamenti in cui la differenza è che il primo tende a recuperare le informazioni della stessa domanda mentre il secondo recupera le informazioni frammentate, la memoria basata sui grafi invece non ha recuperato le informazioni dai suoi archi utili mentre

la cronologia completa che viene usata come controllo diagnostico non può essere classificabile in questo confronto perché ha accesso all'intero storico senza effettuare dei recuperi;

- nell'indicare se le risposte sono fedeli alle informazioni in possesso, Turn-level RAG è l'architettura che ottiene dei buoni risultati mentre la cronologia completa è nettamente superiore a tutte e tre le architetture perché ha accesso all'intero storico;
- nell'astenersi alle domande di cui non si possiedono le fonti necessarie nel risponderle, tutte le strategie di memoria si astengono correttamente.

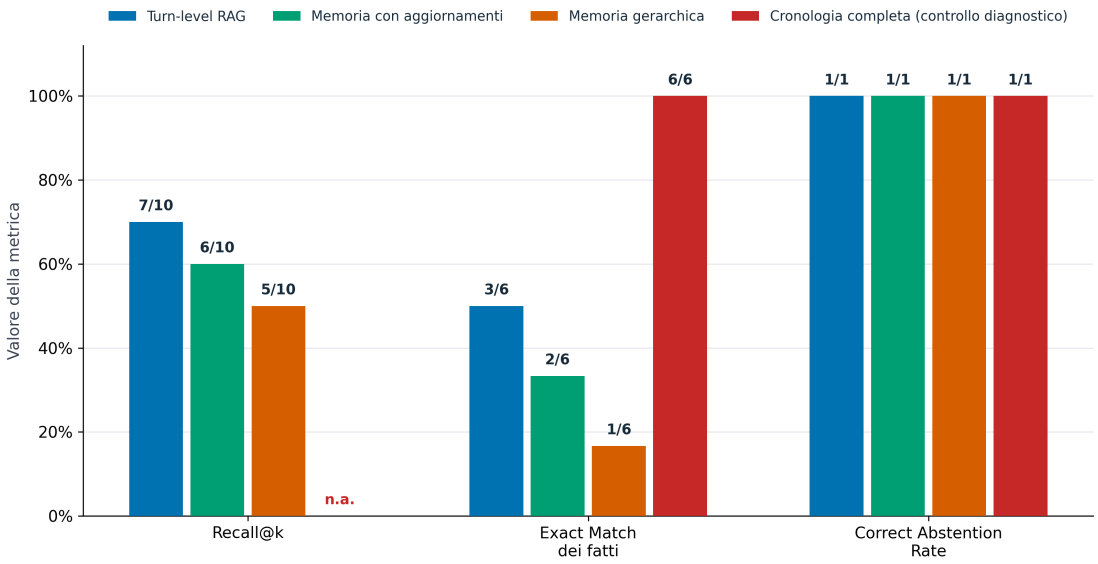


Figura 6.4: Confronto nello scenario 5

Dal seguente grafico 6.4 dell'ultimo confronto si può osservare che:

- nel recuperare le informazioni, Turn-level RAG è l'architettura che ottiene più informazioni seguita da poi la memoria con gli aggiornamenti e la memoria gerarchica

mentre la cronologia completa che viene usata come controllo diagnostico non può essere classificabile in questo confronto perché ha accesso all'intero storico senza effettuare dei recuperi;

- nell'indicare se le risposte sono fedeli alle informazioni in possesso, è l'architettura Turn-level RAG recupera il maggior numero di informazioni, seguita dalla memoria con gli aggiornamenti e la memoria gerarchica. La cronologia completa ottiene risultati superiori, poiché ha accesso all'intero storico.

- nell'astenersi alle domande di cui non si possiedono le fonti necessarie nel risponderle, entrambe le strategie di memoria si astengono correttamente.

Se si utilizza il limite di 200 token in questo scenario, la cronologia completa supera il numero di utilizzo dei token risultando quindi più dispendiosa.

6.2.2 Risposta alla domanda di ricerca

Quindi osservando grafici e i risultati ottenuti dal calcolo delle metriche si può quindi esprimere che non esiste al momento l'architettura di memoria migliore, la cronologia completa permette di gestire al meglio la memoria tuttavia comporta un aumento del costo computazionale.

6.3 Risultati della terza domanda di ricerca

Tabella 6.2: Confronto tra memoria separata e memoria condivisa

Metrica	**Memoria separata**	**Memoria condivisa**
Risposte corrette	120/120 (100%)	120/120 (100%)
Informazioni recuperate	120/120 (100%)	120/120 (100%)
Messaggi contaminanti	0/240 (0%)	17/240 (7,1%)

Attraverso i risultati nella presente tabella 6.2 mostrano che il recupero delle informazioni pertinenti e correttezza delle risposte vengono influenzate in base alle diverse configurazioni degli scenari analizzati. La memoria condivisa aumenta il recupero di informazioni non pertinenti, ma negli scenari analizzati tale contaminazione non determina un degrado dell'accuratezza finale. Questo suggerisce una certa robustezza del modello al rumore nel contesto.

6.4 Risultati della quarta domanda di ricerca

Tabella 6.3: Risultati della continuità della memoria dopo il cambio di modello.

Metrica	**Memoria separata**	**Memoria condivisa**
Risultato	Memoria separata S6–S9	Memoria condivisa S1–S9
Fatti storici presenti nel contesto	1/7	6/7
Recall@k — Gemma	4/4	4/4
Recall@k — Llama	2/4	4/4
Fatti recenti nei controlli	1/3	0/3

I risultati riportati nella tabella 6.3 mostrano che nella configurazione memoria separata il nuovo modello ha un accesso limitato alle informazioni presenti nelle sessioni precedenti.

Questa limitazione incide sul recupero delle informazioni storiche anche se non è identico ad entrambi i modelli.

Nel caso della memoria condivisa aumenta la disponibilità delle informazioni favorendo ad entrambi i modelli il recupero delle informazioni necessarie. Il cambio del modello non determina necessariamente una perdita di continuità: il fattore determinante è l'accesso alla memoria condivisa.

6.5 Risultati della quinta domanda di ricerca

Tabella 6.4: Confronto tra Gemma 3 4B e Llama 3.2 3B

Condizione	**Gemma 3 4B**	**Llama 3.2 3B**	**Differenza**
Contesto sufficiente: Exact Match	1613/2016 (80,0%)	1604/2016 (79,6%)	$+0,4$ punti percentuali
Contesto insufficiente: Correct Abstention	1721/2016 (85,4%)	1527/2016 (75,7%)	$+9,6$ punti percentuali

Attraverso i dati della presente tabella 6.4, si può osservare che entrambi i modelli nel caso del contesto sufficiente hanno valori di Exact Match molto simili, con una differenza di 0,4 punti percentuali. Nel caso del contesto insufficiente invece entrambi i modelli riescono ad astenersi correttamente nel momento in cui non hanno informazioni sufficienti, con una differenza di 9,6 punti percentuali a favore di Gemma.

Quando il contesto è sufficiente, la scelta tra i due modelli considerati incide poco sull'Exact Match. In condizioni di informazione insufficiente emerge invece una differenza più marcata nella capacità di astensione, indicando una maggiore sensibilità di questa proprietà al modello utilizzato.

Conclusioni

Il problema analizzato nella presente tesi è quello dello studio della memoria conversazionale dei modelli linguistici. Gli aspetti principali del presente studio sono stati: l'accesso alle informazioni passate, quale strategia di memorizzazione risulta la migliore, quanto le informazioni secondarie distraggono il modello linguistico dal contesto principale, quanto un nuovo modello linguistico riesce a mantenere i vincoli imposti dal precedente modello condividendone la memoria e quanto i risultati dipendono dalla scelta del modello linguistico da usare.

Utilizzo dei sistemi di memoria esterni favoriscono alla migliore generazione di risposte del modello perché permettono di recuperare le informazioni passate e di avere delle diverse strategie di memorizzazione che possono essere ottime sulla base della situazione che si deve affrontare perché non basta solo conservare le informazioni ma anche saper usare la giusta strategia nel momento migliore.

Attraverso gli esperimenti nella presente tesi si può concludere che:

1. dal confronto tra i diversi livelli di disponibilità della memoria si può osservare che la correttezza delle risposte del modello aumenta perché ha accesso a più informazioni presenti nello storico;

2. non esiste al momento una strategia di memorizzazione che risulti la migliore perché

ogni strategia ha i suoi pregi e difetti;

3. La memoria condivisa aumenta il recupero di informazioni non pertinenti, ma negli scenari analizzati tale contaminazione non determina un degrado dell'accuratezza finale. Questo suggerisce una certa robustezza del modello al rumore nel contesto;

4. nell'esperimento sul cambio di modello tra la memoria separata e quella condivisa è emerso che nella memoria separata avviene un peggioramento delle risposte perché il nuovo modello ha un accesso limitato alle informazioni delle sessioni precedenti. Nella memoria condivisa non avviene lo stesso peggioramento. Il cambio del modello non determina necessariamente una perdita di continuità: il fattore determinante è l'accesso alla memoria condivisa;

5. quando il contesto è sufficiente, la scelta tra i due modelli considerati incide poco sull'Exact Match. In condizioni di informazione insufficiente emerge invece una differenza più marcata nella capacità di astensione, indicando una maggiore sensibilità di questa proprietà al modello utilizzato.

I principali limiti della presente tesi sono:

- sono stati utilizzati dei scenari sintetici per avere meno variabili da affrontare ed avere gli esperimenti più controllati;
- sono stati utilizzati tre modelli linguistici che sono pochi;
- c'è stato un numero limitato di sessioni.

I possibili sviluppi futuri includerebbero:

- possibilità di utilizzare dei scenari reali in cui contengono delle discussioni aziendali vere permettendo quindi analizzare la spontainetà degli utenti;

- utilizzare per gli esperimenti con altri modelli linguistici sia della stessa famiglia che di famiglie diverse;

- avere un numero illimitato di sessioni in cui si utilizza un grafico in cui mostra come varia il recupero delle informazioni sulla base della lunghezza della conversazione;

Appendice

Comandi da eseguire nel terminale

I seguenti comandi sono i comandi più importanti per utilizzare il framework della presente tesi.

Primo esperimento

Per calcolare e verificare le metriche:

```
python3 scripts/summarize_evaluation.py --check
```

Secondo esperimento

Per rigenerare i grafici:

```
.venv-grafici/bin/python scripts/plot_rq2_results.py --all
```

Terzo esperimento

Per verificare i risultati senza chiamare un modello:

```
python3 scripts/rq3/validate_rq3_sc07_inputs.py
```

Quarto esperimento

Per verificare i risultati senza chiamare un modello:

```
python3 scripts/rq4/validate_rq4_inputs.py --phase generation
```

Quinto esperimento

Per verificare i dati caricati in ingresso senza chiamare un modello:

```
python3 scripts/rq3/validate_rq5_inputs.py \
  --config data/rq3/config/rq5_sc06_v1_1.json \
  --inputs-dir data/rq3/sc06_rq5_v1_1
```

Bibliografia

[1] Zeyu Zhang, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Quanyu Dai, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A Survey on the Memory Mechanism of Large Language Model Based Agents. *arXiv preprint arXiv:2404.13501*, 2024. Disponibile su: `https://doi.org/10.48550/arXiv.2404.13501`.

[2] Shervin Minaee, Tomas Mikolov, Narjes Nikzad, Meysam Chenaghlu, Richard Socher, Xavier Amatriain, and Jianfeng Gao. Large Language Models: A Survey. *arXiv preprint arXiv:2402.06196*, 2024. Disponibile su: `https://doi.org/10.48550/arXiv.2402.06196`.

[3] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. Curran Associates, Inc., 2020. Disponibile su: `https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html`.

[4] Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory.

Proceedings of the Thirteenth International Conference on Learning Representations (ICLR), 2025. Disponibile su: https://doi.org/10.48550/arXiv.2410.10813.

[5] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building Production-Ready AI Agents with Scalable Long-Term Memory. *arXiv preprint arXiv:2504.19413*, 2025. Disponibile su: https://doi.org/10.48550/arXiv.2504.19413.

[6] Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic Memory for LLM Agents. *arXiv preprint arXiv:2502.12110*, 2025. Disponibile su: https://doi.org/10.48550/arXiv.2502.12110.

[7] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as Operating Systems. *arXiv preprint arXiv:2310.08560v2*, 2024. Disponibile su: https://arxiv.org/abs/2310.08560v2.

[8] Nishanth Madhusudhan, Sathwik Tejaswi Madhusudhan, Vikas Yadav, and Masoud Hashemi. Do LLMs know when to NOT answer? investigating abstention abilities of large language models. In Owen Rambow, Leo Wanner, Marianna Apidianaki, Hend Al-Khalifa, Barbara Di Eugenio, and Steven Schockaert, editors, *Proceedings of the 31st International Conference on Computational Linguistics*, pages 9329–9345, Abu Dhabi, UAE, January 2025. Association for Computational Linguistics. Disponibile su: https://aclanthology.org/2025.coling-main.627/.

[9] Lingrui Mei, Jiayu Yao, Yuyao Ge, Yiwei Wang, Baolong Bi, Yujun Cai, Jiazhi Liu, Mingyu Li, Zhong-Zhi Li, Duzhen Zhang, Chenlin Zhou, Jiayi Mao, Tianze Xia, Jiafeng Guo, and Shenghua Liu. A Survey of Context Engineering for Large

Language Models. *arXiv preprint arXiv:2507.13334*, 2025. Disponibile su: https://doi.org/10.48550/arXiv.2507.13334.

[10] Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating Very Long-Term Conversational Memory of LLM Agents. *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. Disponibile su: https://doi.org/10.18653/v1/2024.acl-long.747.

[11] Anthropic. Claude. *Sito ufficiale di Anthropic*, s.d. Disponibile su: https://claude.com/product/overview.

[12] John D. Hunter. Matplotlib: A 2d graphics environment. *Computing in Science & Engineering*, 9(3):90–95, 2007. Disponibile su: https://ieeexplore.ieee.org/document/4160265.

[13] GitHub. GitHub Advisory Database. Archivio pubblico di advisory di sicurezza, s.d. Disponibile su: https://github.com/github/advisory-database.

[14] Shahul Es, Jithin James, Luis Espinosa Anke, and Steven Schockaert. RAGAs: Automated evaluation of retrieval augmented generation. In Nikolaos Aletras and Orphee De Clercq, editors, *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, pages 150–158, St. Julians, Malta, March 2024. Association for Computational Linguistics. Disponibile su: https://doi.org/10.18653/v1/2024.eacl-demo.16.

[15] Meta. Llama 3.2 model card, 2024. Disponibile su: https://github.com/meta-llama/llama-models/blob/main/models/llama3_2/MODEL_CARD.md.

[16] Gemma Team. Gemma 3. 2025. Disponibile su: https://doi.org/10.48550/arXiv.2503.19786.

[17] Ollama. Ollama. Repository ufficiale del software, s.d. Disponibile su: https://github.com/ollama/ollama.

[18] Verizon RISK Team. The VERIS Community Database (VCDB). VERIS Framework, s.d. Disponibile su: https://verisframework.org/vcdb.html.

[19] Verizon RISK Team e contributori. VERIS Community Database (VCDB). Repository di dati pubblicato su GitHub, 2026. Versione del 4 agosto 2026, identificata dal commit 230cf22b56a481dd1a994b21e4d94c59e2bccea9. Disponibile su: https://github.com/vz-risk/VCDB/tree/230cf22b56a481dd1a994b21e4d94c59e2bccea9.